6/04

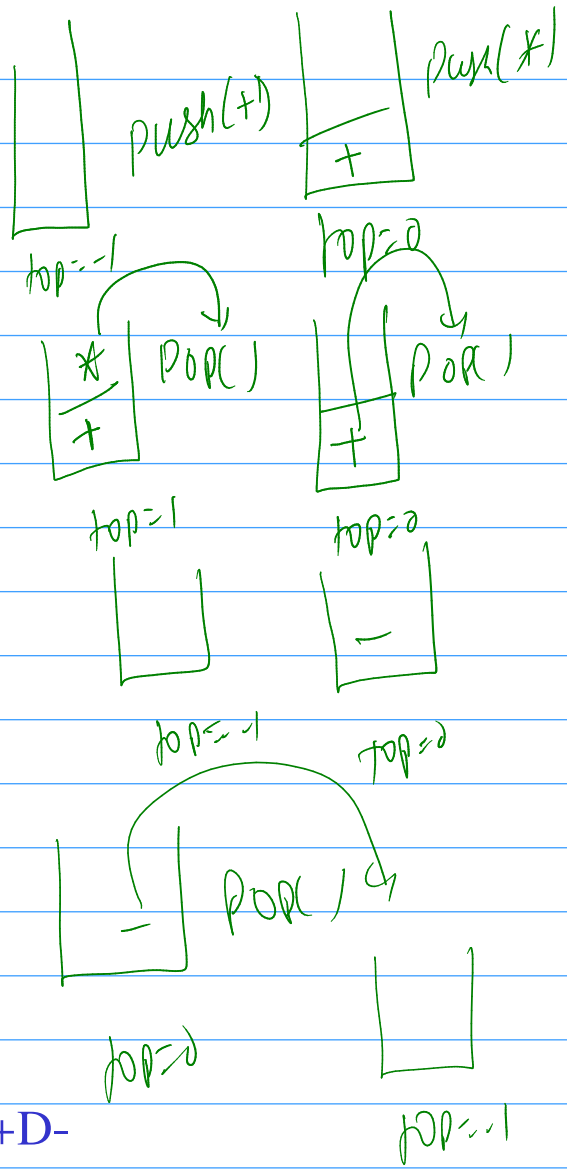
infix to postfix conversion

char e[]="A+B*C-D"

exp	action	stack	PE
A	Add to PE		A
+	Push +	+	A
B	Add to PE	+	AB
*	Push *	+ *	AB
C	Add to PE	+*	ABC
-	Pop	+	ABC*
	Pop		ABC*+
	push -	-	
D	AddtoPE	-	ABC*+D
\0	terminate the loop		

pop the stack unitil it becomes empty

ABC*+D-



(A+B*C)/(D-E+F)

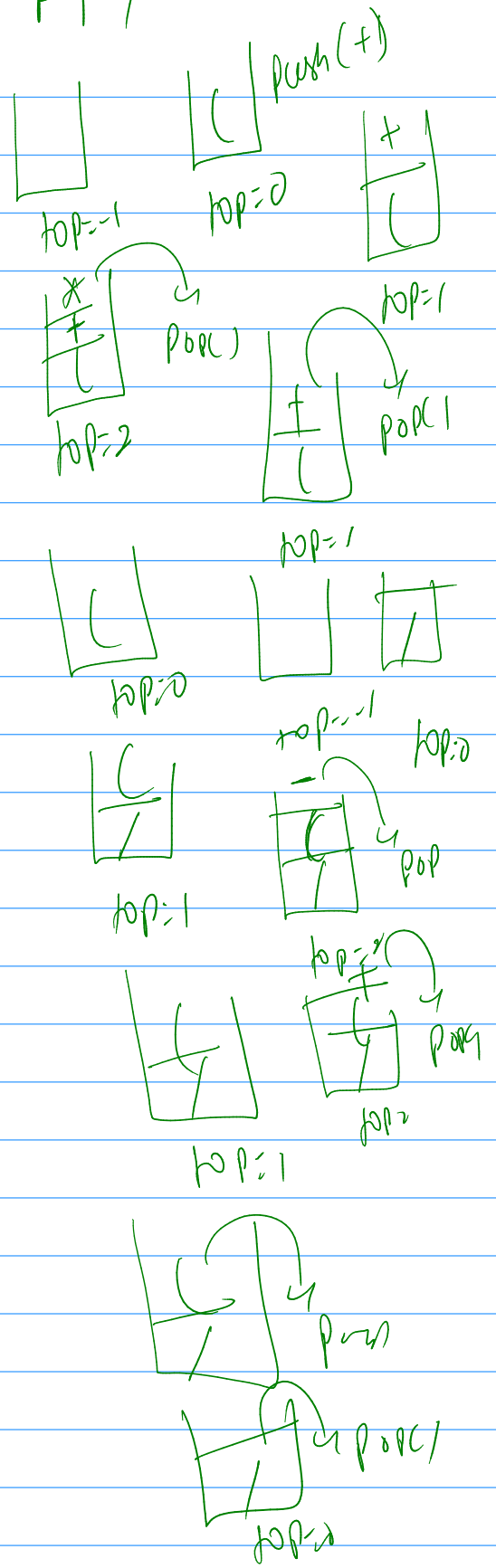
ABC*+DE-F+/

exp	action	stack	PE
(	push	(	
A	add to PE		A
+	Push +	(+	A
B	Add to PE	(+	AB
*	Push *	(+*	AB
C	Add to PE	(+*	ABC
)	Pop	(+	ABC*
	Pop	(	ABC*+
	pop return (termiante		
/	Push /	/	ABC*+
(	push (	/(	ABC*+
D	Add to PE	/(	ABC*+D
-	Push	/(-	
E	Add to PE	/(-	ABC*+DE
+	Pop	/(	ABC*+DE-
	push +	/(+	ABC*+DE-
F	Add to PE	/(+	ABC*+DE-F
)	Pop	/(	ABC*+DE-F+
	Pop	/	
\0	terminate the loop		

pop the stack unitil it becomes empty

Pop()

ABC*+DE-F+/



Algorithm for Evalauting postfix epxression

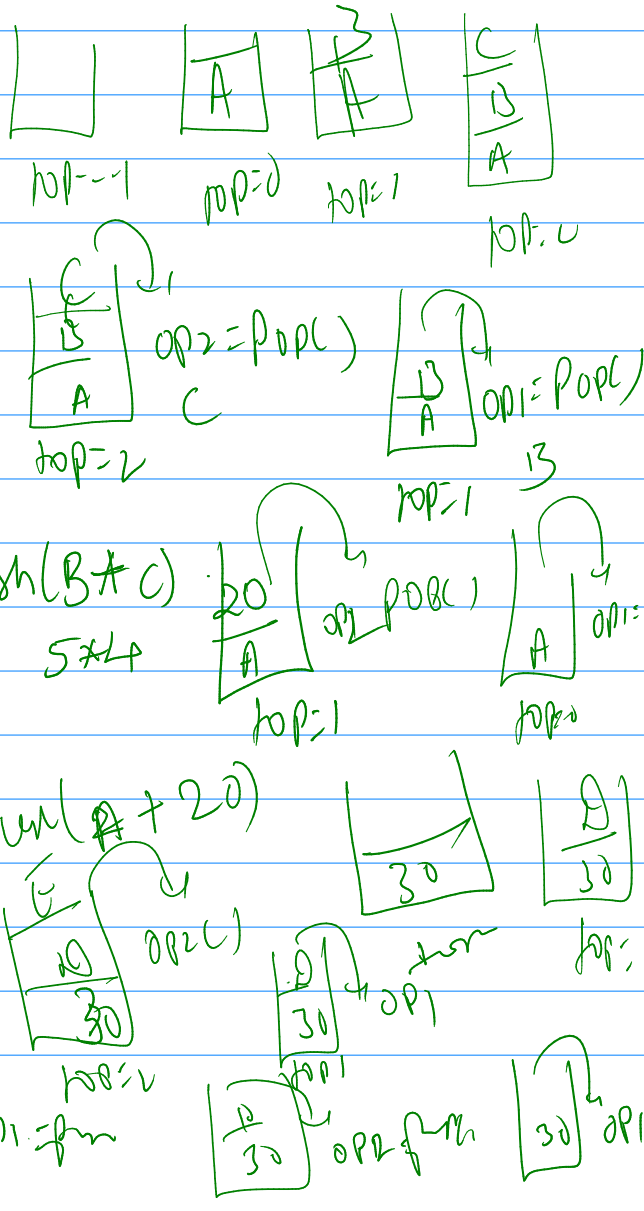
1. traverse from left to right
2. if current element isopreand push on to stack
3. if current element is operator pop the twice , apply the operator on poped datas and push the result.

Note : first poped data should become second operand and second pope data should become first operand

4. repeat above steps unitil we reach end of the expression.
5. When we reach end of the expression result is on the top of th stack

(A+B*C)/(D-E+F) A=10 B=5 C=4 D=15 E=7 F=2 (3)

Exp	Action	Stack
A	push A	A
B	push B	AB
C	push C	ABC
*	Pop twice push result	A 20
+	Pop twice Push result	30
D	Push	30 D
E	push	30 D E
-	Pop twice push result	30 8
F	push	30 8 F
+	Pop twice push result	30 10
/	Pop twice p[ush result	3
\0		



A+B*C -----> ABC*+ ------> (A+(B*C))

5 + 10 * 2 5 10 2 * + ---> 25

Queue

Queue is a linear data structure which follows first in first out(fifo)

Queue has two ends, front and rear end

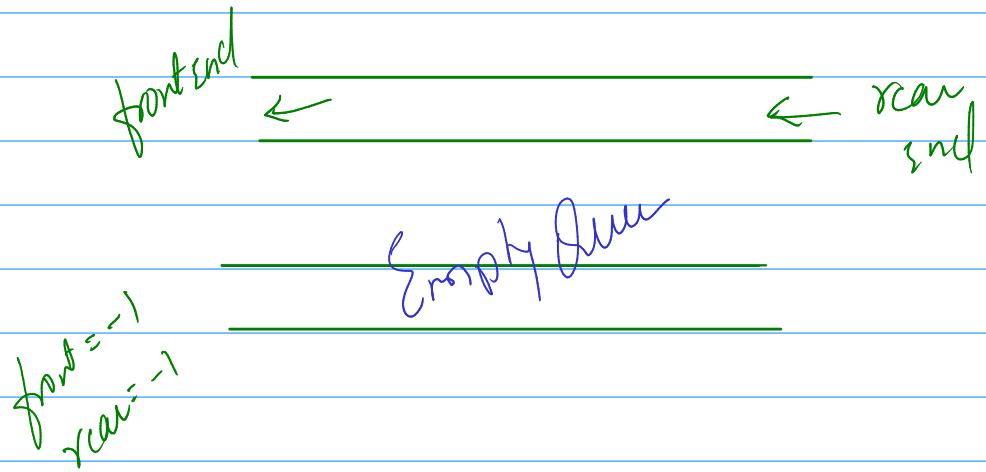
operations of Queue

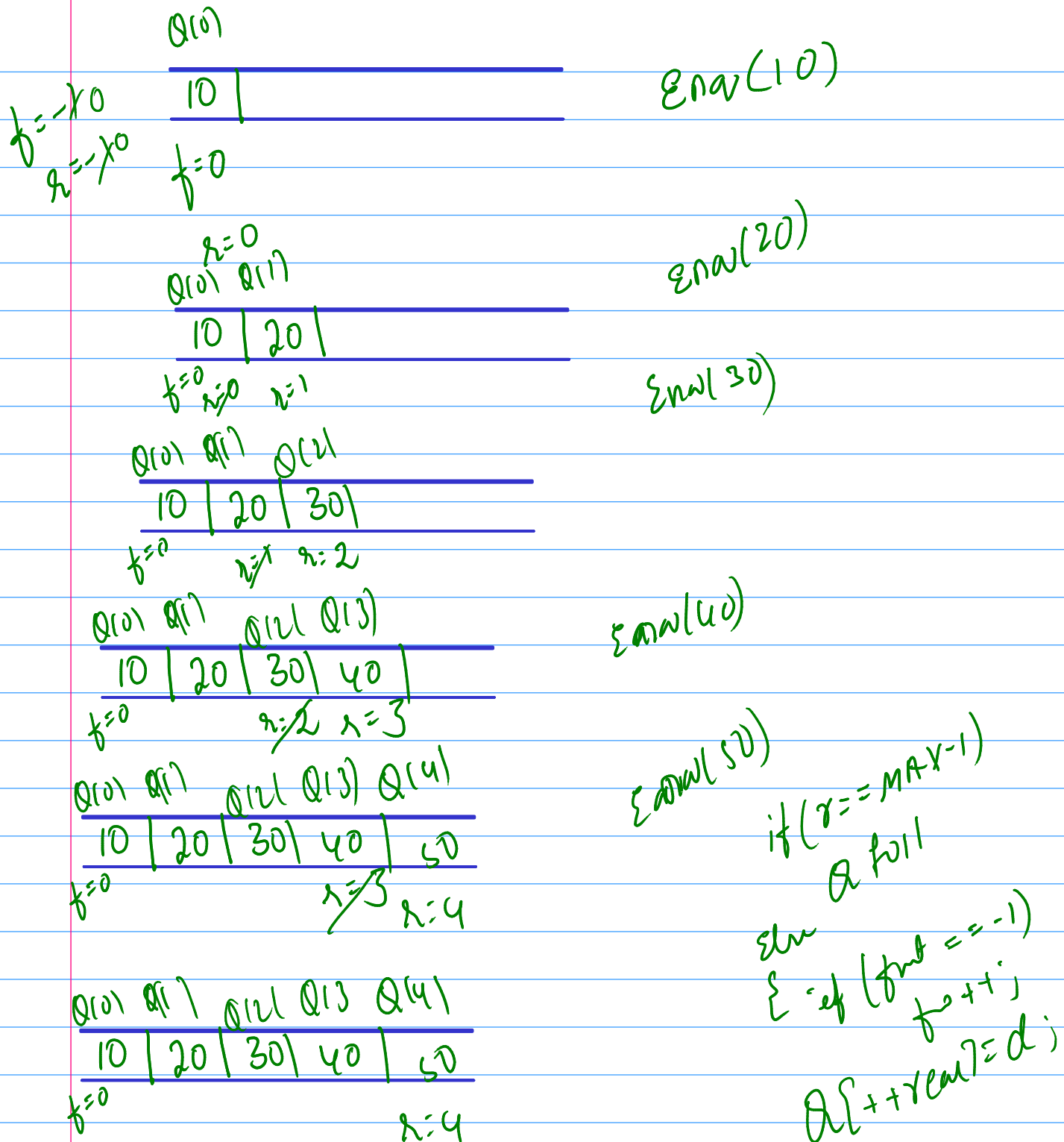
1. enqueue : inserting the data in queue is called enqueue
   we always insert at rear end
2. dequeue : deleting the data from queue is called dequeue
   we always delete from front end

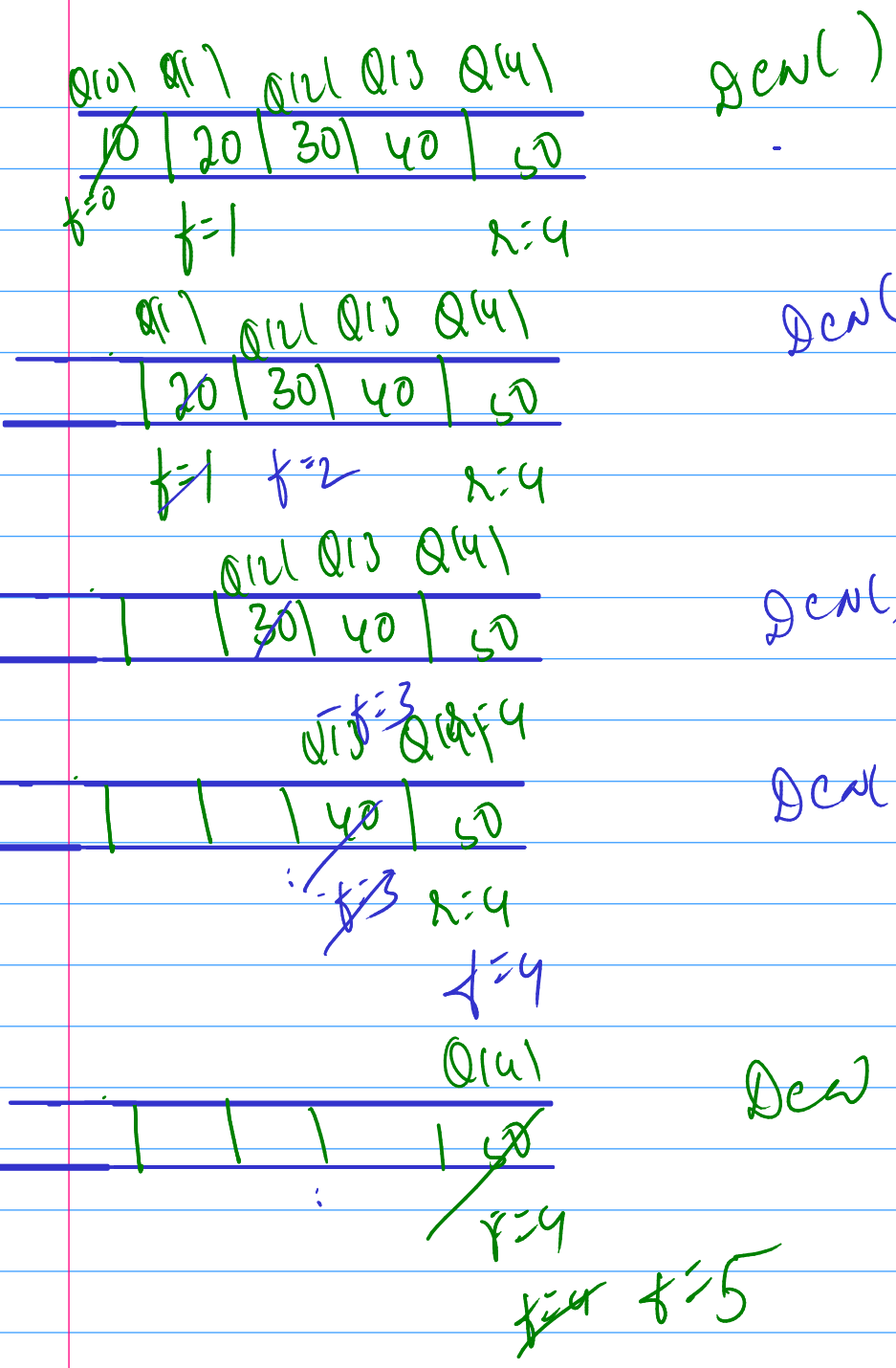
Queue can implemented with arrays and link list.

Queue with arrays

front and rear acts index to array




Q(0)
10
enq(10)
f=-10
r=-10
f=0
r=0
Q(0) Q(1)
10 | 20
enq(20)
f=0
r=0
r=1
Q(0) Q(1) Q(2)
10 | 20 | 30
enq(30)
f=0
r=1
r=2
Q(0) Q(1) Q(2) Q(3)
10 | 20 | 30 | 40
enq(40)
f=0
r=2
r=3
Q(0) Q(1) Q(2) Q(3) Q(4)
10 | 20 | 30 | 40 | 50
enq(50)
f=0
r=3
r=4
Q(0) Q(1) Q(2) Q(3) Q(4)
10 | 20 | 30 | 40 | 50
f=0
r=4
if(r==MAX-1)
Q full
else
if(front==-1)
f++;
Q[++rear]=d;


Q[0] Q[1] Q[2] Q[3] Q[4]
10 20 30 40 50
f=0
f=1
r=4
deq()
Q[1] Q[2] Q[3] Q[4]
20 30 40 50
f=1 f=2
r=4
deq()
Q[2] Q[3] Q[4]
30 40 50
f=3
deq()
Q[3] Q[4]
40 50
f=3
r=4
f=4
Q[4]
50
f=4
Dcw
f=5